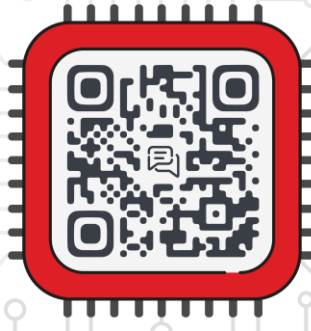




4



14



1680 sp

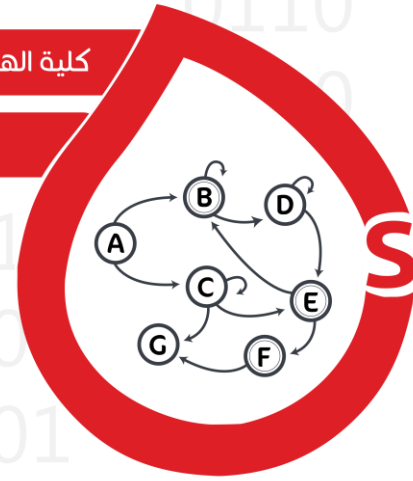
نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Regular Expressions

د. هبة حاتم



RB Informatics; 01/05/2025

اللغات الصورية

Regular Expressions (التعابير المنتظمة)

هي وسيلة جبرية للتعبير عن الأنماط لأي String، بحيث إذا كنت تريد تصميم أوتوماتا ما، تكون هذه الأوتوماتا تقبل هذا الـ String

Regular Expressions تصف بشكل محدد ودقيق الـ Regular Languages

نذكر أن: الـ Regular Languages هي عبارة عن اللغات التي تقبلها الـ Finite State Automata بأنواعها الثلاثة (DFA, NFA, ϵ -NFA) وبالتالي فإن أي Regular Expression هو عبارة عن:

Regular Language التي يعرفها هذا الـ Regular Expression

كنا نعرف الـ Language بأحرف، لكن الآن سنقول:

If E is a regular expression $\rightarrow L(E)$ is the language it defines

إذا كان (E) هو التعبير المنتظم، فإن $L(E)$ هو اللغة التي يعرفها هذا التعبير.

الفرق بين الـ Regular Expression والـ Finite Automata

- الـ FA: هي مودل للـ machine، يعني هي أقرب للـ machine $\leftarrow$ هذه الـ machine هي عبارة عن مجموعة من الـ states والـ transitions التي تقبل Input String والخرج يكون إما Accept أو Reject
- الـ Regular Expression: تكون عبارة عن تعبير قواعدي / لغوي وبالتالي هي أقرب للـ Human Language من الأوتوماتا، إذاً: هي تشبه الطريقة التي يساق فيها البرنامج / أشبه بالقواعد التي يساق فيها البرنامج (More Program Syntax Facts)
- ولكن:
- كل من الـ FA والـ Regular Expression يرتبط بالـ Regular Languages
- فالـ FA: يقبل الـ Regular Languages
- والـ RE: يقبل الـ Regular Languages

إن Regular Expressions يتم تعريفه باستخدام الـ Symbols الموجودة بأبجدية اللغة باستخدام العمليات المنظمة (Regular Operations)

لنتذكر معاً: RE يعرف لغة ← هو يستخدم ثلاث عمليات منتظمة:

1. Union / Or:

تعبّر عن الـ Or حيث L, M حالتين:

$L \cup M$:

هي state عناصرها إما تنتمي لـ L أو لـ M أول لـ كليهما

2. Concatenation / And:

في حال A, B حالتين فإن:

$$A \cdot B = xy \quad ; \quad x \in A, y \in B$$

أي النتيجة هي عبارة عن state كل symbol فيها عبارة عن مزيج symbol ينتمي لـ state الأولى و symbol ينتمي لـ state الثانية

3. Star *:

وهي عملية تطبق على مجموعة واحدة فقط، تعطيني كافة الاحتمالات من مزيج الـ Symbols المؤلفة لهذه المجموعة متضمنة الإبسلون ϵ

4. Parentheses ():

تستخدم من أجل الـ Grouping

Languages Operators

1. Union of two languages:

- $L \cup M$ = all strings that are either in L or in M
- Note: A union of two languages produces another language
- نعبر عن الاجتماع بالرمز "+"
- Example: $0 + 1$
- هو عبارة عن regular expression يمثل إما الـ Zero Symbol أو الـ One Symbol

2. Concatenation of two languages:

- $L \cdot M / L^{\circ} M / LM = xy$
 $s.t / ; x \in L, y \in M$
- اسمها The dot operator
- Example:

$$(0 + 1)0 \rightarrow \{00, 10\}$$

$$(0 + 1)1 \rightarrow \{01, 11\}$$

- يعني عملياً سيعطيني two symbols كل واحد منهم ينتهي ب 1 وطوله 2
- نتيجة ال Concatenation هو لغة جديدة ولكنها أيضاً Regular Language

سنتعلم لاحقاً أن:

اللغات المنتظمة مغلقة (closed) أمام ال Concatenation

$$(0 + 1) (0 + \varepsilon) \rightarrow \{ 00, 0, 10, 1 \}$$

تقرأ: Zero or One Concatenated Zero or ε

وهي:

قد تبدأ ب 0 أو قد تبدأ ب 1

وقد تنتهي ب 0 أو قد لا تنتهي بشيء

(هناك ε أعطتني احتمالية أن تخرج حالة مكونة من Symbol واحد فقط)

Kleene Star Kleene Closure (The * Operator)

انطلقت فكرة ال Kleen star من ال step by step للخطوة الأولى

مجموعة منتهية ولا تتألف إلا من ε $L^0 = \{ \varepsilon \}$

عملياً هي ال set نفسها $L^1 = \{ w \mid \text{for some } w \in L \}$

يعني إذا عندي a, b $L^1 = \{ a, b \}$

$$L^2 = \{ w_1 w_2 \mid w_1 \in L, w_2 \in L \text{ (duplicates allowed)} \}$$

L^2 تسمح لي بمررة واحدة من ال Concatenation، إما العنصر مع نفسه، أو العنصر من جيرانه، ولكن مرة واحدة فقط

$$L^3 = \{ \dots \dots \}$$

تسمح مرتين بال Concatenation

جار العنصر مع العنصر مرة ثم مع نفسه مرة	bab	العنصر مع نفسه مرتين	aaa
جار العنصر مع نفسه مرة ثم مع العنصر مرة	bba	جار العنصر مع نفسه مرتين	bbb
العنصر مع جاره مرتين	abb	العنصر مع نفسه مرة ثم مع جاره مرة	aab
جار العنصر مع العنصر مرتين	baa	العنصر مع جاره مرة ثم مع نفسه مرة	aba

$$L^1 = \{ w_1 w_2 \dots w_i \mid \text{all } w' \text{ chosen are } \in L \text{ (duplicates allowed)} \}$$

ويكون عدد الاحتمالات 2^i

Note: the choice of each w_i is independent

$$\rightarrow L^* = \bigcup_{i \geq 0} L^i \text{ (arbitrary number of concatenation)}$$

عدد اعتباطي / كيفي من التسلسلات، ولذلك هي مجموعة غير منتهية (infinite set)

حيث L : تشير إلى عدد الـ strings التي نريد تطبيق الـ Concatenation عليهم من أجل إنتاج مجموعة strings داخل L

■ Example:

$$\begin{aligned} \text{let } L &= \{ 1, 00 \} \\ L^0 &= \{ \varepsilon \} \\ L^1 &= \{ 1, 00 \} \\ L^2 &= \{ 11, 100, 001, 0000 \} \\ L^3 &= \{ 111, 1100, 1001, 10000, 000000, 00001, 00100, 00001 \} \\ L^* &= L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots \end{aligned}$$

Special Note / Formal Definition

L^* is an infinite set iff (if and only if) $|L| \geq 1$ and $L \neq \{ \varepsilon \}$

Ex:

$$L = \{ 1 \} \rightarrow L^* = \{ \varepsilon, 1, 11, 111, \dots \}$$

وبالتالي L هي infinite set

حتى تكون L مجموعة غير منتهية يجب أن يتحقق الشرطين معاً:

1. طول L أكبر أو يساوي الواحد

2. $L \neq \{ \varepsilon \}$

فيما عدا ذلك فهي مجموعة منتهية (Finite Set)

$$\text{if } L = \{ \varepsilon \}, \text{ then } L^* = \{ \varepsilon \}$$

$$\text{if } L = \phi, \text{ then } L^* = \{ \varepsilon \}$$

وهذه قواعد أساسية سنستخدمها في استقرارات مستقبلية

وهاتان الحالتان تكون فيهما المجموعة منتهية

Σ^* denotes the set of all words over an alphabet Σ

Σ^* تمثل مجموعة كافة الكلمات عبر أبجدية معينة، وبالتالي صيغة مختصرة لقول أنه يوجد لغة اعتباطية عبر Σ (سيغما) هي:

$$L \subseteq \Sigma^*$$

لقد تحدثنا وقلنا أن الـ Regular Expressions يتم بناؤها انطلاقاً من الـ Symbols الخاصة بالأبجدية مع الـ Regular

($\cup, *, \text{Concatenation}$) Operators

What is the precedence of arranging these operations in regular expressions?

Highest to Lowest:

$* \text{ Star } \rightarrow \cdot \rightarrow + / \text{ Union }$

لكن في حال تواجد الأقواس فهي للـ Grouping وبالتالي يكون لها الأولوية العليا، أي في حال وجود Parentheses فهذا يعني تغيير في الـ Priority

■ Example: $01^* + 1 = (0 \cdot (1)^*) + 1$

$01^* + 1$: الأولوية العليا للـ $*$ بأخذ 1^* :

$1^* = \{ \epsilon, 1, 11, 111, \dots \}$

$0 \cdot 1^*$: تمثل كافة الـ Strings الممكنة عبر الـ 0 و 1 والتي تبدأ بـ 0 وقد تنتهي بـ 1 وقد لا تنتهي بـ 1

0: عبارة عن رمز مقبول في هذه اللغة، لماذا؟

لأن:

1^* : قد يكون ϵ والـ 1 في القسم الثاني (أي $01^* + 1$) قد لا يكون موجود، لأن الـ $+$ عبارة عن or وبالتالي قد يكون موجود وقد لا يكون موجود

- من المهم جداً معرفة القواعد والأولويات في العمليات لمعرفة الجواب الصحيح
- يفضل في حال مرور أي تعبير جبري بسيط يفضل حفظه من أجل تسهيل الحل في الامتحان

RE's: Definition 1

أي رمز داخل الأبجدية ببعض المرجعيات يدعى Terminal، هذا الـ Terminal أو هذا الـ Symbol هو لوحده Regular Expression

Basis 1:

If a is any symbol, then a is a RE, and $L(a) = \{a\}$

- القاعدة الأولى:

أي a ينتمي لـ RL هو عبارة عن RE بحد ذاته، بمعنى $L(a) = \{a\}$ ، أي هي بحد ذاتها لغة قائمة تحتوي على عنصر واحد فقط

Note: $\{a\}$ is the language containing one string, and that string is of length 1

Basis 2:

ϵ is a RE, and $L(\epsilon) = \{\epsilon\}$

Basis 3:

ϕ is a RE, and $L(\phi) = \phi$

الفرق بين ϵ و ϕ : ϕ : هي المجموعة الخالية ϵ : null string، يعني المحرف الفارغ أو السلسلة الفارغة أو العنصر الفارغ

تذكر: أن الابسيلون في $NFA - \epsilon$ أضاف مرونة Flexibility كبيرة في عمل الآلة، وهو الذي يمثل انتقال اللاشيء وليس الانتقال الصفري لأنه في الانتقال الصفري يعني أنه جاء symbol قدره صفر، بينما انتقال الابسيلون هو الانتقال الفارغ، وفي حال كان الانتقال غير ممثل ظاهرياً، فكل حالة لها انتقال فراغ لنفسها، فمثلاً، $q_1 \xrightarrow{\epsilon} q_2$ حتى لو أنها غير ممثلة ظاهرياً

- In other words, any symbol $\in \Sigma$ is a RE
- Definition: The REs over Σ are the Res obtained by applying these rules one or many times
- التعبير المنتظم عبر Σ هو عبارة عن مزيج من التعبيرات المنتظمة التي أولدها انطلاقاً من تطبيق واحدة أو أكثر من هذه القواعد
- انطلاقاً من الـ Bases هذه سنحصل على ثلاثة استقرئات (3 inductions):

Induction 1:

If E_1 and E_2 are regular expressions then $E_1 + E_2$ is another regular expression, and $L(E_1 + E_2) = L(E_1) \cup L(E_2)$

REs are closed under union

أي: إذا تم تطبيق هذه العملية على أي مجموعتين فإن الخرج سيكون مجموعة تنتمي إلى هذه المجموعة

- ماذا يعني مصطلح "مغلقة" في مجموعة مغلقة؟

على سبيل المثال، مجموعة الأعداد الطبيعية هي مجموعة مغلقة أما عملية الجمع، لأن جمع أي عددين طبيعيين هو عدد طبيعي، وغير مغلقة أمام عملية الطرح لأن طرح أي عددين طبيعيين ليس من الضرورة أن يكون عدداً طبيعياً، وهذا الـ Induction 1، أن تكون Res مغلقة أمام الاجتماع $\cup$ (مهم جداً للتحويل من RE إلى FA)

Induction 2:

If E_1 and E_2 are regular expressions, then $E_1 \cdot E_2$ is another regular expression, and $L(E_1 \cdot E_2) = L(E_1) \cdot L(E_2)$

Res are closed under concatenation

- Concatenation: the set of strings $w x$ such that w is in $L(E_1)$ and x is in $L(E_2)$

Induction 3: if E is a RE, and $L(E^*) = (L(E))^*$

The Kleene star of RE is also a RE

- Star = Kleene Star = closure = Kleene closure

■ Example 1:

$$L(01) = \{01\}$$

$L(01)$ هو عبارة عن RE(0) concatenated with RE(1)

$$L(01 + 0) = \{01, 0\}$$

$$L(0(1 + 0)) = \{01, 00\}$$

أولوية الترتيب للأقواس

الأولوية العليا هي لما داخل الأقواس $(0 + 1)$: إما 0 أو 1، لكن الـ concatenation الأولي مع الصفر وبالتالي $\{01, 00\}$

$$L(0^*) = \{\epsilon, 0, 00, 000, \dots\}$$

$$L((0 + 10)^*(\epsilon + 1)) = \text{all strings of 0's and 1's}$$

أولاً:

Without two consecutive 1's $(0 + 10)$

التعبير $(0 + 10)$: يعطي إما 0 أو 10

التعبير $(0 + 10)^*$: يعطي جميع السلاسل بما فيها الفراغة المؤلفة من التكرار العشوائي لـ 0 و 10

التعبير $(\epsilon + 1)$: يعني إما السلسلة فارغة (ϵ) أو الرقم (1)

الآن نربط (concatenation) بين الجزأين:

أي خذ كل السلاسل من $(0 + 10)^*$ وضع في نهايتها 1 أو لا شيء (ϵ) ، أي أن اللغة الناتجة هي: كل السلاسل التي تتكون من تكرار 0 و 10، وقد يكون في نهايتها إما لا شيء أو 1 وبالتالي هي مزيج من الأصفار والواحدات التي لا يظهر فيها واحدتين متتاليتين

يعني النتيجة هي:

$$\{\epsilon, 0, 10, 00, 010, 100, 1010, 001010, \dots\}$$

منتهية إما بـ 1 أو بلا شيء، أي:

$$\{\epsilon, 0, 01, 10, 101, 00, 001, 010, 0101, 100, 1001, 1010, 10101, 001010, 0010101, \dots\}$$

■ Example 2:

Find the RE for the language of all binary strings of length at least 2 and ends in the same symbol

على الأقل طولها 2: يعني على الأقل يجب أن نرى كلمة تحوي حرفين

كلمة على الأقل: تعني ضمناً أنه يجب التفكير بموضوع الـ Star

تبدأ وتنتهي بنفس الرمز: لدينا احتمالين لا ثالث لهما، إما تبدأ وتنتهي بـ 0، أو تبدأ وتنتهي بـ 1
"at least 2" تعني:

بالحالة الأولى يجب أن تحتوي $(0 + 1)$

بالحالة الثانية يجب أن تحتوي $(0 + 1)$

ويكونان بكل حالة $(0 + 1)^*$

ولكن لماذا * ؟

لأنه حتى لو كانت $\varepsilon = (0 + 1)^*$

$$|\varepsilon| = 1$$

أيضاً بالحالة الثانية:

$$0\varepsilon 0 = 00$$

وبين الحالتين اجتماع "Union" لأن النتيجة الكلية هي اجتماع هاتين التيجتين، وبالتالي فإن الـ Regular Expression الذي يعبر عن الصيغة اللفظية للغة هو:

$$0(0 + 1)^* 0 + 1(0 + 1)^* 1$$

تذكروا دائماً أن:

Star *: ممكن أن تمثل أصغر قطعة ε ، وممكن أن تمثل أطول قطعة (اللانهاية هي أكثر شيء)

Example 3:

لدي هذا الـ RE:

$$((0 + 1)^* 1 + \varepsilon) (00)^* 00$$

أريد التعبير لفظياً عن اللغة التي يعبر عنها الـ Regular Expression أولاً:

نبدأ من الصفرين: أي مهما يكن قبلهما فهذان الصفران من الإجباري أن يكونوا موجودين نعود لنفكر من البداية:

$(0 + 1)^*$ جميع السلاسل بما فيها الفارغة المؤلفة من التكرار العشوائي لـ 0 و 1

Concatenated with 1: هذه السلاسل المنتهية بـ 1

ε : أو السلسلة فارغة

$(00)^*$ Concatenated with: يعني قد تكون ε وقد تكون عدد غير منتهٍ من مزيج الأصفار الثنائية للتعبير سأقول:

أن هذا الـ RE يصف اللغة التالية:

هي لغة تحوي binary strings تنتهي بعدد غير صفري من الأصفار الزوجية (من الإجباري أن تحوي على صفرين بالنهاية على الأقل)

This RE is for the set of all binary strings that end with an even non-zero number of 0's

مثلاً: 00 أو 00 00

Concatenation يعني (إجباري أن يكون موجوداً)

Example 4:

$L = \{ w \mid w \text{ is a binary string which does not contain two consecutive 0's or two consecutive 1's anywhere} \}$

كتابة RE الذي يعبر عن اللغة التالية:

اللغة عبارة عن مجموعة الكلمات التي تحوي أصفار وواحدات التي لا تحتوي صفرين متتاليين مع بعض، ولا واحدتين متتاليين مع بعض

1. نضع جميع الحالات
2. نقوم بعمل RE لكل حالة
3. نقوم بعمل اجتماع لكل الحالات وبالتالي نحصل على الـ RE الذي يعبر عن هذه اللغة

لا أريد two consecutive 0's أو two consecutive 1's

وهذه اللغة هي over any string، يعني الأطوال يجب التفكير بها، قد تكون طويلة وقد تكون صغيرة لذا يجب التفكير بالـ Kleene Star

وظيفتي عدم وضع zero after zero أو one after one

كيف؟

أولاً:

- التعبير عن أصغر واحدة طول ← *
- الصفر هو عبارة عن سيغما ولكن طوله 1
- ولكن ابسيلون عبارة عن null string

Ex: $w = 01010101$ is in L , while $w = 10010$ is not in L

Four cases for w_i :

- Case A: w starts with 0 and $|w|$ is even
- Case B: w starts with 1 and $|w|$ is even
- Case C: w starts with 0 and $|w|$ is odd
- Case D: w starts with 1 and $|w|$ is odd
-

Regular Expressions for the four cases:

- Case A: $(01)^*$
- Case B: $(10)^*$
- Case C: $0(10)^*$
- Case D: $1(01)^*$

Since L is the union of all 4 cases:

- Reg Exp for $L = (01)^* + (10)^* + 0(10)^* + 1(01)^*$
- If we introduce ε then the regular expression can be simplified to:

Reg Exp for $L: (\varepsilon + 1)(01)^* (\varepsilon + 0)$

Or:

Reg Exp for $L: (\varepsilon + 0)(10)^* (\varepsilon + 1)$

- إدخال الـ ε أعطى الكثير من المرونة والسرعة في الحل

Algebraic Laws of Regular Expressions

■ القوانين الجبرية في التعابير المنتظمة:

تعتبر القوانين الجبرية مهمة جداً، لأنه بالاعتماد عليها نستطيع التحويل من الـ Finite Automata إلى Regular Exps وبالعكس من Regular Exps إلى Finite Automata
سنعتمد هذه العمليات الرياضية كحجر الأساس لكثير من الأشياء لاحقاً

Commutative (الخاصية التبادلية):

$$E + F = F + E$$

Associative (الخاصية الترابطية / التجميعية):

$$(E + F) + G = E + (F + G)$$

$$(EF)G = E(FG)$$

Identity (خاصية الهوية):

$$E + \phi = E$$

$$\varepsilon E = E\varepsilon = E$$

من خاصية الـ Identity نستنتج الكثير من الأشياء يتم تعميمها على الـ Kleene Star / Kleene Closure سنراها بعد قليل

Annihilator (الخاصية التدميرية):

$$\phi E = E\phi = \phi$$

Distributive (الخاصية التوزيعية):

$$E(F + G) = EF + EG$$

$$(F + G)E = FE + GE$$

Idempotent (الخاصية غير ذات الفائدة / العاجزة):

$$E + E = E$$

Involving Kleene Closures:

$$(E^*)^* = E^*$$

$$E^+ = EE^*$$

$$E^*E^* = E^*$$

$$s.t: E^+: is E^* without \varepsilon$$

$$\phi^* = \varepsilon$$

$$\varepsilon^* = \varepsilon$$

$$(ER)^*E = E(RE)^*$$

$$(E + R)^* = (E^*R^*)^* = (E^* + R^*)^*$$

Arden's Theorem:

سنحدث عنها، نبرهنها ثم نطبقها، فيها سيتم التحويل من DFA, non-DFA إلى RE

■ نظرية آردن:

إذا كان عندي:

P is a RE but without ε

أيضاً:

Q is a RE that might contain ε , and it might be ε

فإذا كان لدي:

$$R = RQ + P$$

$$R = Q + RP$$

ما هو الـ RE الذي تمثله R؟

$$R = PQ^*$$

$$R = QP^*$$

وهذا الحل هو حل وحيد (unique)

هذا سيفيدنا بإيجاد الـ Regular Expressions للأوتومات المختلفة

الآن سنثبت البرهان:

Proof Arden's Theorem:

$$R = Q + RP$$

سنعوض R بنفسها كل مرة

$$R = Q + (Q + RP)P$$

$$R = Q + QP + RPP$$

$$R = Q + QP + (Q + RP)PP$$

$$R = Q + QP + QP^2 + RP^3$$

وهكذا، لنعوض بعد مرة والنتيجة:

$$R = Q + QP + QP^2 + QP^3 + \dots$$

$$R = Q (\varepsilon + P + P^2 + P^3 + \dots)$$

$$R = QP^*$$

وبالتالي فإن:

$$R = QP^*$$

$$R = Q + RP \leftarrow \text{نفسها}$$

Thus, $R = QP^*$ is the unique solution of the equation $R = Q + RP$

- Note: Arden's Theorem is used to convert given finite automata to regular expression

Example:

Find the regular expression of q_1 and q_2 using Arden's Theorem

$$q_1 = q_1 0 + \varepsilon = \varepsilon 0^* = 0^*$$

$$q_2 = q_1 1 + q_2 0$$

$$q_2 = 0^*1 + q_20$$

$$q_2 = 0^*10^*$$

Applications of Arden's Theorem

1. Regular Expression Derivation:

اشتقاق الـ RE (وليس المشتق وإنما استنباطها)

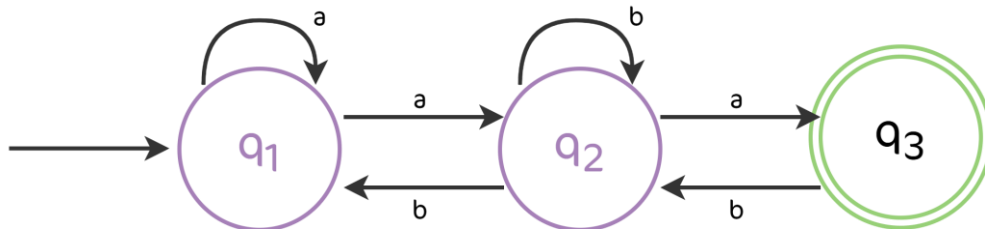
حتى المعادلات التي تحتوي على Regular Expressions، تفيدني بتحويل الأوتومات، لكن كيف؟
أن يكون لدي Automaton، وأعبر عن كل حالة من الحالات بمزيج من مجموعة العمليات الداخلة لها، انطلاقاً من الـ Symbols، وعندما أعبر عنه سنستخدم Arden's Theorem لإيجاد الـ RE المكافئة لهذه الأوتوماتا المعينة

2. Automata Conversion:

Convert deterministic or non-deterministic finite automata (DFA / NFA) into equivalent regular expressions

NFA to RE:

الـ RE الخاص بالـ Automata هو الـ RE الخاص بالـ FS (Finite State)، وفي حال كان لدي أكثر من Accept State نأخذ الـ RE لها جميعاً



كيف سنقوم بالتعبير عن كل حالة؟ بالانطلاق من الـ Final State

$$q_3 = q_2 \cdot a \rightarrow \textcircled{1}$$

يجب أن نجد q_2

$$q_2 = q_1 \cdot a + q_2 \cdot b + q_3 \cdot b \rightarrow \textcircled{2}$$

يجب أن نعبر عن q_1 لكي نستطيع أن نعوض

$$q_1 = \varepsilon + q_1 \cdot a + q_2 \cdot b \rightarrow \textcircled{3}$$

3 معادلات تعبر عن كل الـ States، لا، وإنما تعبر عن الـ Finite State

الهدف: إمكانية تطبيق Arden's Theorem لاستخراج الـ RE

سنقوم بسلسلة من التعويضات:

$$\begin{aligned} &\rightarrow q_3 = q_2 \cdot a \\ &= (q_1 \cdot a + q_2 \cdot b + q_3 \cdot b) a \\ &= q_1 aa + q_2 ba + q_3 b^* a \end{aligned} \textcircled{4}$$

نبحث عن q_1, q_2 بشكل أفضل

$$\rightarrow q_2 = q_1 \cdot a + q_2 \cdot b + q_3 \cdot b$$

نعوض المعادلة 1 لـ q_3

$$\begin{aligned} &= q_1 a + q_2 b + (q_2 a) b \\ &= q_1 a + q_2 b + q_2 a b \\ q_2 &= q_1 a + q_2 (b + ab) \\ q_2 &= (q_1 a) (b + ab)^* \rightarrow \textcircled{5} \end{aligned}$$

الآن نبحث عن q_1 بدلالة a, b

$$q_1 = \varepsilon + q_1 a + q_2 b$$

من 5 نعوض q_2 :

$$\begin{aligned} q_1 &= \varepsilon + q_1 a + ((q_1 a) (b^2 + ab)^*) b \\ q_1 &= \varepsilon + q_1 (a + a(b + ab)^*) b \\ q_1 &= \varepsilon ((a + a(b + ab)^*) b)^* \\ q_1 &= ((a + a(b + ab)^*) b)^* \rightarrow \textcircled{6} \end{aligned}$$

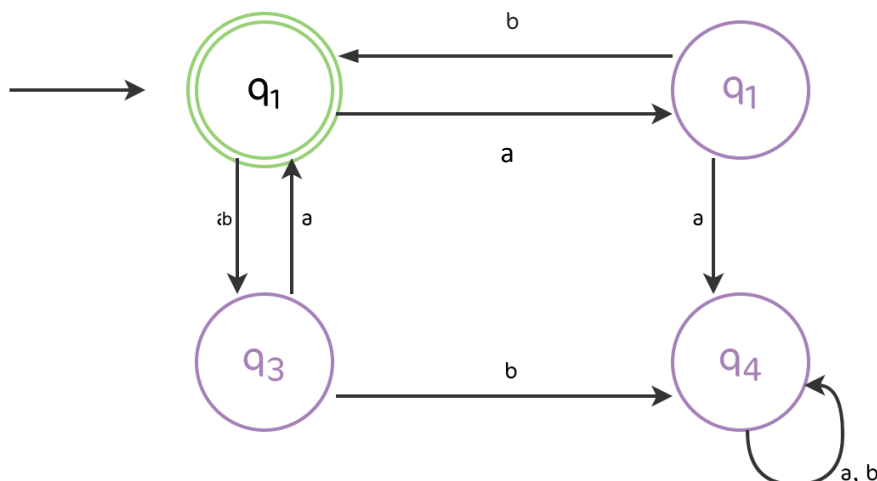
Final State:

$$\begin{aligned} q_3 &= q_2 a \\ &= q_1 a (b + ab)^* a \\ q_3 &= ((a + a(b + ab)^*) b)^* a (b + ab)^* a \end{aligned}$$

وهذه هو الـ RE المطلوب للتعبير عن الـ NFA

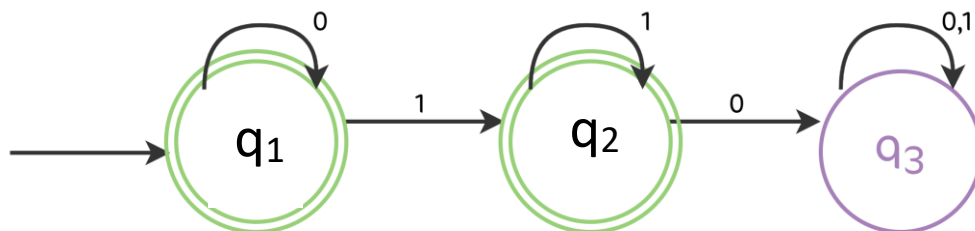
DFA to RE

- الـ Trap State عندما لا يكون هناك لفظ مباشر عند هذه الحالة
- عدم وجود لفظ صريح إذا أخذت State معينة symbol معين فإن هذا ليس مؤشر على عدم الحتمية، عدم الحتمية أن يكون لـ State معينة أكثر من انتقال عند Symbol واحد فقط، وبالتالي إذا رأينا State لم يأتها أحد رموز اللغة أو ليس معبر عنها ظاهرياً بشيء فهذا يعني أنه ذاهب إلى Trap State



$$\begin{aligned} q_1 &= \varepsilon + q_2 b + q_3 a \\ q_2 &= q_1 a \\ q_3 &= q_1 b \\ q_1 &= \varepsilon + q_1 ab + q_1 ba \\ q_1 &= \varepsilon + q_1 (ab + ba) \\ q_1 &= \varepsilon (ab + ba)^* \quad (RE) \end{aligned}$$

Find the regular expression for the following DFA:



بحسب RE للـ two final states وبعمل union بينهم

Final state q_1 :

$$q_1 = \varepsilon + q_1 0 \rightarrow q_1 = \varepsilon 0^* = 0^*$$

$$q_2 = q_1 1 + q_2 1$$

$$q_2 = 0^* 1 + q_2 1 = 0^* 1 (1^*)$$

$$RE = RE_1 \cup RE_2$$

$$= 0^* + 0^* 1 (1^*)$$

$$= 0^* (\varepsilon + 1 (1^*))$$

$$RE = 0^* 1^*$$